Guia rápido para uso do cluster: o *hpcdmc*

1 - Introdução

Em 17 de janeiro de 2014 o novo cluster ficou pronto para uso. Dependendo da demanda de cada usuário, obviamente será necessária a instalação de algum software adicional - mas isso não deve atrapalhar o funcionamento e uso simultâneo por outros usuários desde já

O conjunto atual está constituído por 7 servidores bi-processados, 1 switch e 2 no-breaks de 10kVa, montados como um cluster beowulf para computação de alto desempenho. Ou seja, existe um servidor mestre, responsável pelo gerenciamento dos processos submetidos aos nós - especificamente, a configuração inicial montada disponibiliza 6 nós.

Cada nó escravo constitui-se uma solução 2U Supermicro, com 2 x Intel Xeon E5-2690, 4x8GB RAM DDR3 ECC Kingston e 2x500GB (RAID 1) em disco SATA 3 Seagate. Já o nó mestre também é uma solução 2U Supermicro, com 2 x Intel Xeon E5-2620, 4x8GB RAM DDR3 ECC Kingston e 8 x 2TB em discos SATA 3 Enterprise divididos em 2 arranjos (um arranjo RAID 1 para o sistema local e outro RAID 6, que disponibiliza o armazenamento distribuido do cluster).

Em outras palavras, cada nó disponibiliza 32 processadores (dos quais 2 são reservados para tarefas administrativas e 30 para submissão de processos) e cerca de 1GB de memória RAM por processador. Já o servidor mestre é responsável por disponibilizar uma área de armazenamento compartilhada (visivel por todos os nós) de aproximadamente 8TB. Em anexo, segue relação detalhada do hardware.

No que se refere ao software, foi usado apenas software livre, sendo o sistema operacional o CentOS 6.5.
O nó mestre (master) gerencia as contas de usuário por NIS e usa NFS para disponibilizar a partição /home aos nós escravos (que são chamados de n01, n02, n03, n04, n05, n06).

A comunicação interna entre as máquinas faz-se por uma rede LAN Gigabit Ethernet sendo o acesso externo via o IP 200.145.184.171 exclusivamente via SSH. Aliás, todo acesso (interno e externo) faz-se exclusivamente por SSH com chave pública/privada.

Do ponto de vista operacional, é o nó mestre que, via o software SLURM, gerenciará os recursos do sistema através de uma fila. Portante, o usuário interage com o nó mestre e este, por sua vez, é o único responsável por interagir com os nós (isto é, é o nó mestre que atribui as tarefas aos nós, e não o usuário). Em outras palavras, o nó mestre deverá ser usado pelos usuários exclusivamente para:

- acesso ao cluster (via SSH através do IP 200.145.184.171).
- compilação dos códigos dos usuários (recomenda-se que todos programas/códigos submetido ao cluster sejam compilados usando as bibliotecas instaladas no nó mestre).
- preparação de scripts.
- armazenamento de dados de entrada e de saída de trabalhos relacionados ao cluster.
- e, por fim, a submissão de trabalhos ao sistema exclusivamente através da fila.

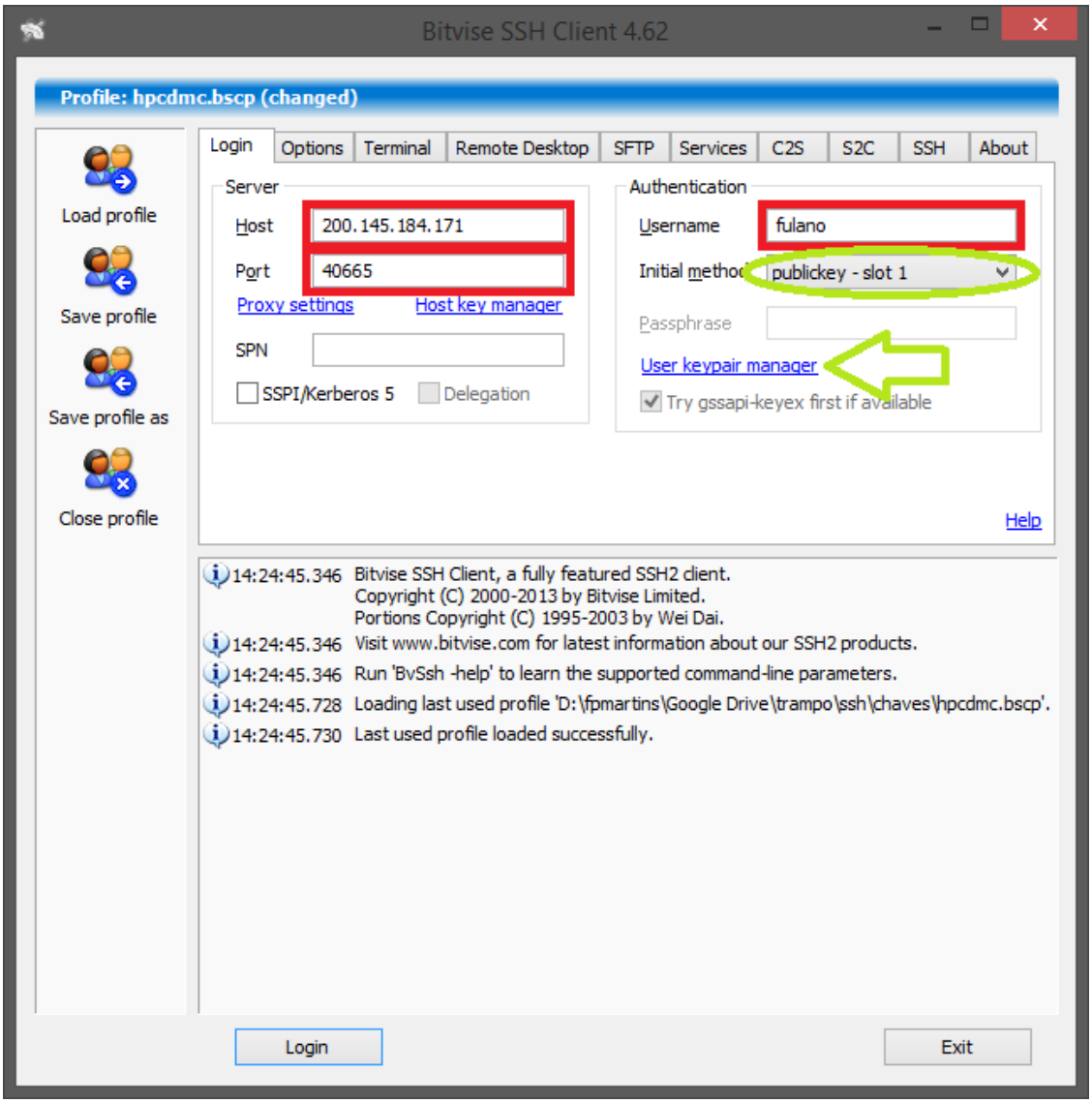
Basicamente, todo trabalho (*job*) é submetido ao cluster através da fila gerenciada pelo SLURM, a qual, a cada submissão, verifica os recursos disponíveis no cluster (isto é, aquilo que já não está em uso por

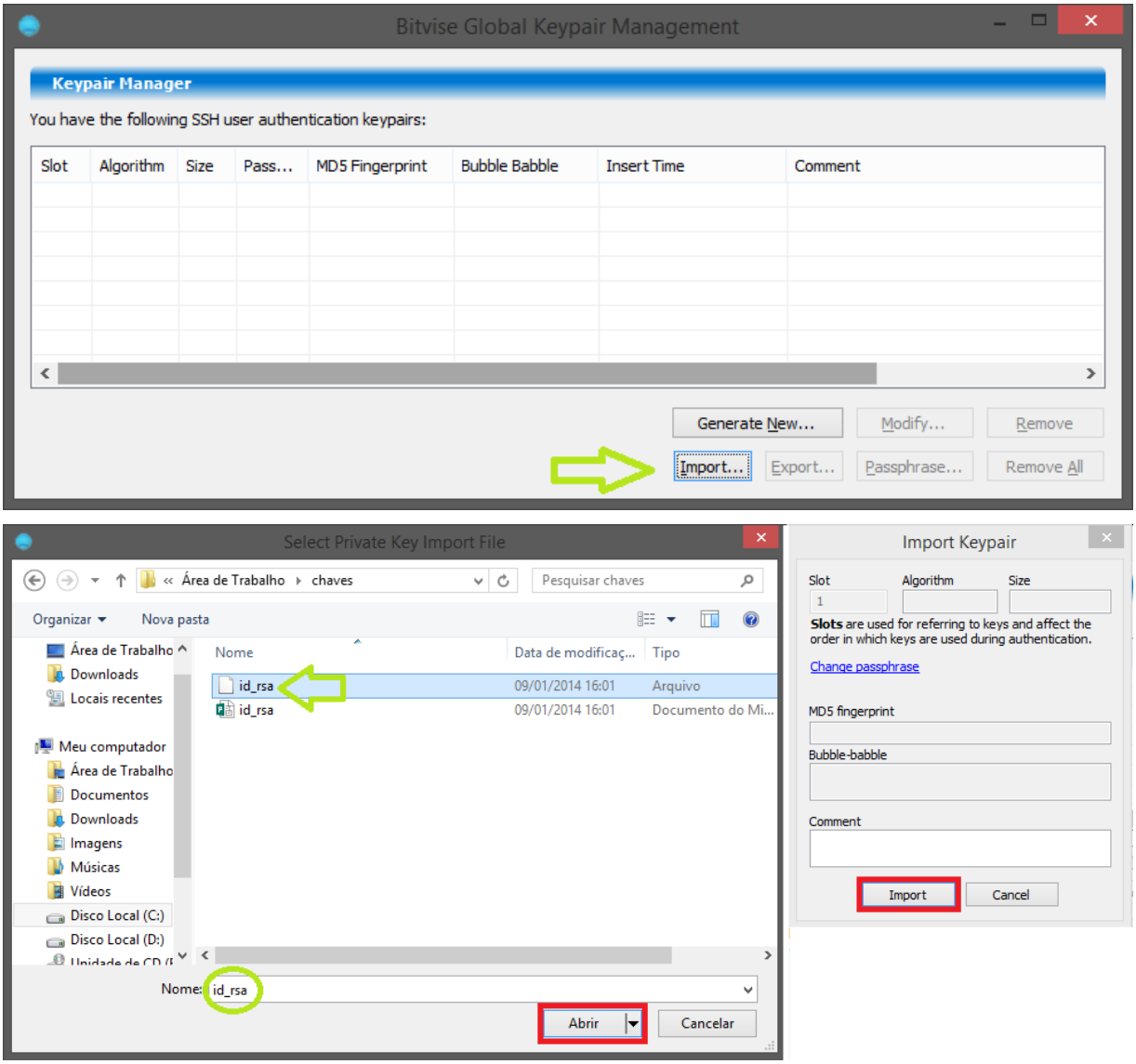
outros usuários) e as exigências de processamento exigidas pelo novo *job*. Assim que as exigências do novo *job* estão disponíveis, ele entra em execução.

2 - Acesso externo ao cluster

O acesso externo ao cluster será feito, exclusivamente, através do IP 200.145.184.171 (171d.fct.unesp.br) por SSH na porta 40665 com chaves pública/privada. Ao receber uma conta de usuário, o mesmo receberá dois arquivos: id_rsa (chave privada) id_rsa.pub (chave pública que foi instalada no cluster).

Se o usuário usa alguma versão do Windows em sua máquina local, recomendamos que o acesso seja feito utilizando o Bitvise como cliente SSH (http://www.bitvise.com/ssh-client-download). A instalação do par de chaves e acesso podem ser feitos, neste caso, para o usuário fictício “fulano”, conforme telas a seguir:





Portanto, após a importação do par de chaves, o acesso será feito através do botão “Login” na tela inicial.

Por outro Lado, se o usuário “fulano” quer se conectar ao cluster usando uma máquina local onde está instalado algum sistema baseado em UNIX (Linux, Mac OSX, FreeBSD), ele deve copiar os arquivos id_rsa e id_rsa.pub para o diretório .ssh, localizado em seu home. Isso pode ser conseguido pelos comandos:

```
[fulano@notebook] $ cp <onde esta o arquivo/id_rsa> ~/.ssh/
[fulano@notebook] $ cp <onde esta o arquivo/id_rsa.pub> ~/.ssh/
[fulano@notebook] $ chmod -R 700 ~/.ssh
```

Por fim, para conectar:

```
[fulano@notebook] $ ssh -p 40665 fulano@200.145.184.171
```

Note que não será solicitada senha para acesso ao cluster, portanto, o usuário deve tomar cuidado com o acesso a seu par de chaves, pois, com ele, qualquer pessoa consegue ter acesso a sua conta no cluster. Assim, em caso de perda dessas chaves (como na perda do notebook, por exemplo), deve-se solicitar ao administrador do cluster que as chaves sejam canceladas e substituidas.

3 - Submissão de trabalhos no cluster

Conforme explicado anteriormente, todo processo de submissão de trabalhos no cluster deve ser feito ao nó mestre, e o software gerenciador da fila agendará a execução do trabalho para assim que houver disponibilidade de recursos nos nós.

Para isso, após efetuar obter acesso via SSH ao cluster, o usuário submeterá seus trabalhos através do comando:

```
[fulano@hpcdmc ~]$ sbatch script.batch
```

onde script.batch é um arquivo em texto plano com diretivas básicas ao gerenciador da fila, informando ao mesmo quais recursos serão necessários e o que deverá ser executado por algum nó. A seguir, em "Código 1", temos um modelo para script.batch que pode ser adaptado para diversas situações, as quais vão depender das características e recursos específicos exigidos pelo software do usuário. Perceba que linhas iniciadas por # são linhas de comentário, exceto quando iniciadas por #SBATCH, que são linhas que definem diretivas ao SLURM. Portanto, as linhas de comentário podem ser suprimidas em uma situação prática de uso do cluster, estando as mesmas aqui por mera conveniência didática.

Após cada trabalho submetido, um número é informado, indicando seu status na fila histórica de trabalhos: esse número é o *jobid* do trabalho, e será utilizado para manipular o job

4 - Submissão de trabalhos no cluster

Uma vez submetidos, o usuário pode precisar interagir com os jobs, no mínimo de duas maneiras: para ver seu status (se está rodando, terminou ou teve execução abortada) ou para abortar a simulação. Para tais tarefas, existem alguns comandos básicos oferecidos pelo SLURM, resumidos na tabela abaixo:

finalidade do comando	**comando no terminal**
mostrar a fila de trabalhos	`squeue` `qstat`
cancelar um *job*	`scancel <jobid>` `qdel <jobid>`
submeter um *job*	`sbatch <nome_script.batch>`
resumir a disponibilidade dos nós	`sinfo` `pbsnodes`

Mais detalhes sobre esses comandos, como parâmetros que podem priorizar a exibição de determinadas informações, podem ser obtidos via o comando man:

```
[fulano@hpcdmc ~]$ man squeue
```

Código 1

```
#!/bin/bash

# nome do job
#SBATCH -J ABOBRINHA

# diretório de trabalho, onde ficam dados de entrada/saida, executáveis e/ou scripts
#SBATCH -D /home/fulano/algumaPasta/algumDiretorio

# Nome do usuário
#SBATCH -A fulano

# Nome da fila: no momento, os nós do cluster possuem apenas um tipo de processador
# Assim, a politica adotada usa o nome da fila para definir o tipo de máquina que vai rodar o job
# Veja que se define o tipo de nó, mas não qual nó em específico será usado
#SBATCH -p XE52690


# Numero de nós e processadores exigidos pelo job: no caso, 1 nó
#SBATCH -N 1
# Numero de processadores. No caso, 1 processador.
#SBATCH -n 1
# Número de threads que vai compor o processo
#SBATCH -c 1
# Veja que foram definidas demandas de processamento para a fila, mas não se diz qual nó atenderá a
# demanda - cabe ao SLURM definir isso conforme os parametros e o andamento da fila

# Em caso de pane no nó, este parâmetro indica que o job aguardará intervenção do usuário
# para ser submetido novamente a fila (recomenda-se não alterar esta opção)
#SBATCH --no-requeue

# Usar --share significa que cada nó receberá novos jobs enquanto exisitr capacidade
# ociosa, isto é, o nó será compartilhado por diversos jobs.
# O oposto é --exclusive e implica que o job exigirá um nó exclusivo para sua execução,
# podendo acarretar a subutilização de recursos (recomenda-se não alterar esta opção)
#SBATCH --share

# Tempo limite em h:min:s que um job permanecerá em execução na fila
# Este parâmetro pode ser omitido, deixando o job sem limite de tempo máximo
#SBATCH --time=00:05:00

# Nome dos arquivos com as saídas padrão e de erro
#SBATCH --error=job.%J.err
#SBATCH --output=job.%J.out

# Linha que executará o job, iniciada por srun e seguida pelo comando do usuário
# Exemplo: srun ./simflow_01 dados modelo saidas saida_01 log gnu
srun <comando_do_usuário>
```